Jobber → Google Calendar Sync — Replication Guide

A step-by-step playbook for a person or a Claude agent to stand up a fresh copy of this integration for a new business. Every phase ends with a **Verify** check, so don't start a phase until the one before it passes.

Legend: AGENT = an agent can do it (browser or connector) · HUMAN = the human must do it (sign-in, secret, approval, download)

```
Jobber (visit created/edited/deleted)
    │  webhook: topic + IDs only, HMAC-signed
    ▼
Cloudflare Worker  /webhooks/jobber ── verify signature ── enqueue ── 200 OK (< 1 s)
    │
    ▼  Queue consumer (retries, dead-letter queue)
Jobber GraphQL: visit(id) + job + client + property + assignedUsers
    │  validate: Visit on a Job, scheduled, assigned
    ▼
Google Calendar API (service account) → event on each assignee's calendar
    │
    ▼
Routemize reads the estimator's calendar → availability blocked → job stop can be created
```

Phase 0 — Collect inputs HUMAN

Use the checklist in `01-REQUIREMENTS.md §7`. Decide names now:

Placeholder	Example used in the reference build
`<GITHUB_OWNER>/<REPO>`	`jose24xhr-sys/jobber-gcal-sync`
`<CF_SUBDOMAIN>`	`joseramirez` (from Cloudflare → Workers → Subdomain)
`<WORKER_URL>`	`https://jobber-gcal-sync.<CF_SUBDOMAIN>.workers.dev`
`<GCP_PROJECT>`	`jobber-gcal-sync`
`<SA_EMAIL>`	`jobber-sync@<GCP_PROJECT>.iam.gserviceaccount.com`

Agent rules: never type passwords, API keys or secrets into any field, and never put them in URLs or chat. Ask the human to paste them. Ask before any OAuth/app-install approval, calendar sharing, or file download.

Phase 1 — Code repository

1. AGENT Create a **private** repo `<REPO>` (GitHub → New repository; no README/gitignore/license).
2. AGENT Copy the reference code into it. Either copy the source repo, or push the `jobber-gcal-sync.zip` contents. If an agent pushes from its own workspace, its GitHub app must have access. The safe way: install the **Claude** GitHub app with *Only select repositories* → *`<REPO>`* (HUMAN approves).
3. AGENT Keep `wrangler.toml` placeholders for now; Phase 3 fills them.

Verify: the repo shows `src/`, `test/`, `migrations/`, `wrangler.toml`, `README.md`. Optional: `npm ci && npm test` → **65 passed**.

Phase 2 — Cloudflare data resources

1. AGENT Pick the right Cloudflare account. If there are several, use the one the business wants to own the Worker.
2. AGENT **D1:** create database `jobber_gcal_sync` (location: closest to the business, e.g. ENAM). Copy its **database ID**.
3. AGENT **Schema:** run all of `migrations/0001_init.sql` against it, through the D1 Console in the dashboard or the Cloudflare connector's `d1_database_query`. It creates 5 tables: `jobber_accounts`, `oauth_states`, `user_calendar_map`, `visit_events`, `sync_log`.
4. AGENT **Queues:** create `jobber-visit-sync` and `jobber-visit-sync-dlq` (Workers → Queues → Create Queue; jurisdiction None).
5. AGENT Read the account's workers.dev subdomain (Workers → *Subdomain* page). **Do not change it**: that would move every Worker on the account.

Verify: D1 lists 5 tables; Queues lists 2 queues (status "Inactive" is normal until deploy).

Phase 3 — Configure and deploy the Worker

1. AGENT Edit `wrangler.toml`:
   - `database_id = "<D1 ID>"`
   - `JOBBER_REDIRECT_URI = "<WORKER_URL>/oauth/jobber/callback"`
   - `DEFAULT_TIMEZONE` = the business timezone
   - keep `keep_vars = true` at the top
   - Commit and push.
2. AGENT Cloudflare → Workers & Pages → **Create** → **Continue with GitHub** → pick `<REPO>` → deploy. Defaults are fine: deploy command `npx wrangler deploy`, root `/`.
   - If the repo doesn't appear, the Cloudflare account is linked to a different GitHub account. Choose **Add GitHub account** and HUMAN authorizes Cloudflare's GitHub app for `<GITHUB_OWNER>`. The approval opens in a pop-up the agent can't see, so ask the human to finish it.
3. AGENT Wait for the build: Initializing → Cloning → Installing → Deploying, all green.

Verify: `GET <WORKER_URL>/health` → `ok`.

From now on, **every push to `main` redeploys automatically**, in about 1 minute.

Phase 4 — Jobber app

1. HUMAN Log in at developer.getjobber.com (separate from the normal Jobber login).
2. AGENT Manage Apps → **New**. Fill in:
   - App name, Developer name, **Callback URL** `<WORKER_URL>/oauth/jobber/callback`, **Short description** (required).
   - **Scopes**: toggle ON **Clients**, **Jobs**, **Scheduled Items**, **Users**. Leave **Write** unchecked.
   - **Webhooks**: add 8 rows, all with URL `<WORKER_URL>/webhooks/jobber`: `VISIT_CREATE`, `VISIT_UPDATE`, `VISIT_COMPLETE`, `VISIT_DESTROY`, `JOB_CREATE`, `JOB_UPDATE`, `JOB_DESTROY`, `APP_DISCONNECT`.
     - NOTE The topic picker is a type-ahead. **Click the option**; pressing Enter selects the first item in the full list (`APP_CONNECT`).
   - Refresh token rotation: leave **ON**.
   - **Save App**.
3. AGENT Read back all 8 topic/URL pairs (e.g. from the form's input values) before saving.

Verify: "App updated" toast. The Authorization section shows Client ID and Client Secret.

Phase 5 — Google Cloud

1. AGENT console.cloud.google.com → New project `<GCP_PROJECT>` (no organization is fine).
2. AGENT APIs & Services → Library → **Google Calendar API** → **Enable**.
3. AGENT IAM & Admin → Service accounts → **Create**: name `jobber-sync`. **Skip roles** and close the role picker; none are needed.
4. HUMAN Service account → **Keys** → Add key → Create new key → **JSON**. Downloads a file; keep it private.
   - If key creation is blocked by an org policy (`iam.disableServiceAccountKeyCreation`), a Google admin must allow it for this project.

Verify: the API page shows **Status: Enabled**; the service accounts list shows `<SA_EMAIL>`.

Phase 6 — Secrets and variables

HUMAN Cloudflare → Workers & Pages → `jobber-gcal-sync` → **Settings → Variables and Secrets → + Add** (type **Secret** for all four):

Name	Value
`JOBBER_CLIENT_SECRET`	From the Jobber app

Name	Value
`GOOGLE_SA_PRIVATE_KEY`	In the JSON file, the single line `"private_key": "-----BEGIN PRIVATE KEY-----\nMII…\n-----END PRIVATE KEY-----\n",` copied **whole**. Extra quotes or the label are fine; the code cleans them. Never paste the multi-line PEM: the field keeps only the first 64 characters
`TOKEN_ENCRYPTION_KEY`	Long random value. **Set it once, before connecting Jobber.** Changing it later means reconnecting
`ADMIN_API_TOKEN`	Long random value; save it in a password manager

AGENT Put `JOBBER_CLIENT_ID` and `GOOGLE_SA_CLIENT_EMAIL` in `wrangler.toml` `[vars]` (not secret) and push.

Verify (agent): the Settings page lists all 6 names. `TOKEN_ENCRYPTION_KEY`, `JOBBER_CLIENT_SECRET`, `GOOGLE_SA_PRIVATE_KEY` and `ADMIN_API_TOKEN` must show **"Value encrypted"**. If any shows a plain value, ask the human to edit it and switch the type to **Secret**.

Phase 7 — Connect Jobber

1. **HUMAN** Open `<WORKER_URL>/oauth/jobber/start` → paste `ADMIN_API_TOKEN` in the password box → **Continue**.
2. **HUMAN** In Jobber's consent screen, as a Jobber **admin**: **Allow Access**.
3. The success page shows **"Connected Jobber account `<name>`"** and the **Account ID** (e.g. `Z2lkOi8vSm9iYmVyL0FjY291bnQv…`).

Verify (agent, via D1):

```
SELECT account_id, account_name, status FROM jobber_accounts;   -- status = active
```

Phase 8 — Share calendars and map estimators

1. **HUMAN** Each estimator, or the Workspace admin, opens Google Calendar → Settings → *their calendar* → **Shared with → Add people and groups** → `<SA_EMAIL>` → **Make changes (see private events as free/busy)** → Send.
   - An agent can do this only for calendars owned by the Google account signed in to the browser, and only after the human agrees.
2. **AGENT** / **HUMAN** Open `<WORKER_URL>/admin` → enter `ADMIN_API_TOKEN`. The page lists every Jobber team member. Type each estimator's **Calendar ID** (usually their email) → **Save mappings**. Leave a row blank to not sync that person.
   - Alternative (agent, no token needed): insert into D1 directly. The API user ID = base64 of `gid://Jobber/User/<n>`. `<n>` comes from Jobber web → Settings → Manage Team, whose link `/manage_team/<base64 n>` holds the number base64-encoded.

```
INSERT INTO user_calendar_map (account_id, jobber_user_id, google_calendar_id, label, active,
updated_at)
VALUES ('<ACCOUNT_ID>', '<USER_ENCODED_ID>', 'estimator@company.com', 'Estimator name', 1,
strftime('%s','now')*1000)
ON CONFLICT(account_id, jobber_user_id) DO UPDATE SET google_calendar_id =
excluded.google_calendar_id, active = 1;
```

3. HUMAN Connect the same calendars in **Routemize** for each estimator, if they aren't already.

Verify: `/admin` shows the calendar next to each estimator.

Phase 9 — End-to-end test

1. AGENT In Jobber: **Create → Job** for an existing client, title `TEST - Calendar sync check`, one visit today or tomorrow with start and end times, **assigned** to a mapped estimator, instructions "Safe to delete" → **Save Job**.
2. AGENT Within ~10 s, `/admin` → *Recent activity* shows `created VISIT_CREATE`. A `noop` right after is normal: Jobber sends a second webhook, and it's de-duplicated.
3. AGENT Google Calendar (that estimator) shows "**`<Client> - <Job title>`**" at the right time, with the property address as location and client, job #, instructions and the Jobber link in the description.
4. Then test the rest:

Do in Jobber	Expect in Google Calendar
Reschedule the visit	Same event moves (no duplicate)
Reassign to another mapped person	Event leaves calendar A, appears on calendar B
Assign two people	One event on each calendar
Unschedule the visit	Event removed
Delete the visit / job	Event removed
Create a Request or Quote	Nothing

5. HUMAN In Routemize: the slot is blocked and a job stop can be created from the event.
6. AGENT Delete the TEST job when done; its event is removed automatically.

If nothing appears: see `03-OPERATIONS.md → Troubleshooting`. The Cloudflare **Observability → Events** log shows the exact error per message.

Done criteria

- ☐ `/health` returns `ok`
- ☐ 4 secrets show "Value encrypted"; `keep_vars = true` is in `wrangler.toml`
- ☐ `jobber_accounts.status = active`
- ☐ Every estimator is mapped on `/admin`, and their calendar is shared with `<SA_EMAIL>`

- ☐ Create, reschedule, reassign and delete all behave as in the Phase 9 table
- ☐ The Requests/Quotes test produces no events
- ☐ TEST job deleted